

TAREA ACADEMICA 1

Implementacion DevOps y evidencias de CI/CD

Informe de cumplimiento basado en el repositorio y en ejecuciones reales de GitHub Actions.

AgroSmart Monitor

Aplicacion Java 17 con Maven, pruebas JUnit 5, pipeline automatizado, despliegue simulado y publicacion de artefactos.

Documento preparado el 2 de septiembre de 2026

Repositorio: github.com/AlexitoXD1/agrosmart-monitor

1. Resumen ejecutivo

AgroSmart Monitor simula lecturas de temperatura y humedad de una camara de frio agricola. La aplicacion clasifica cada lectura como NORMAL, WARNING o ALERT y genera un reporte de texto cuando detecta una alerta. El repositorio integra el cambio de codigo con pruebas, empaquetado, despliegue simulado y publicacion del JAR mediante GitHub Actions.

Resultado verificable

La ejecucion #2, asociada al commit 00f4204, fallo en pruebas y bloqueo las etapas posteriores. La correccion del commit 9d77ab7 activo la ejecucion #3, que termino correctamente y publico un artefacto JAR de 6.34 KB.

2. Objetivos y alcance

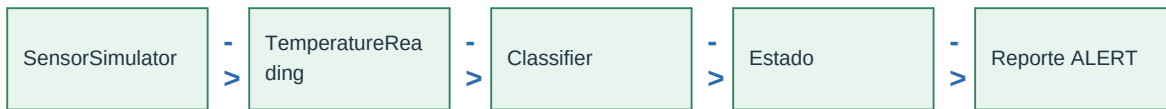
- Implementar un pipeline CI/CD sobre la rama **main**.
- Proteger la regla de negocio con pruebas automatizadas antes del build.
- Relacionar cambio, commit, ejecucion, job, paso, log e impacto.
- Entregar un artefacto recuperable solo cuando la validacion termina correctamente.

Limite del alcance

El despliegue es deliberadamente simulado: copia el JAR a **staging/**. No hay infraestructura de produccion, sensores fisicos ni recomendacion sanitaria real.

3. Solucion implementada

3.1 Arquitectura funcional



Componente	Responsabilidad	Evidencia
SensorSimulator	Genera temperatura de 2 a 12 C y humedad de 45 a 90 %.	Prueba determinista con semilla 42.
TemperatureClassifier	Aplica NORMAL <= 8; WARNING <= 10; ALERT > 10.	Tres pruebas de limites y casos representativos.
AlertReportGenerator	Crea reports/alerta_yyyyMMdd_HHmms.txt ante ALERT.	Invocacion condicionada en Main.
Main	Ejecuta cinco lecturas, imprime estados y reporta alertas.	Punto de entrada declarado en el manifiesto JAR.

3.2 Estructura del repositorio

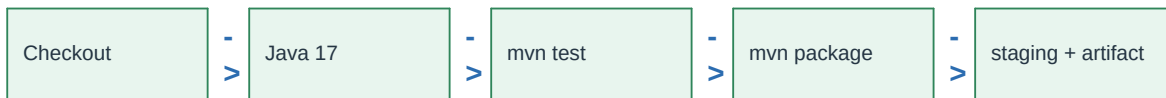
```

agrosmart-monitor/
|-- .github/workflows/ci.yml
|-- pom.xml
|-- README.md
|-- src/main/java/com/agrosmart/
|   |-- Main.java
|   |-- SensorSimulator.java
|   |-- TemperatureReading.java
|   |-- TemperatureClassifier.java
|   |-- SensorStatus.java
|   |-- AlertReportGenerator.java
|-- src/test/java/com/agrosmart/
|   |-- TemperatureClassifierTest.java
|   |-- SensorSimulatorTest.java

```

4. Pipeline CI/CD

El workflow **CI Pipeline - AgroSmart Monitor** se activa con push o pull request sobre main y ejecuta un unico job llamado **build-test-deploy** en Ubuntu.



Paso	Comando o accion	Criterio de exito
Descargar codigo	actions/checkout@v4	Commit disponible en el runner.
Preparar Java	actions/setup-java@v4, Temurin 17, cache Maven	JDK y dependencias preparados.
Pruebas	mvn -B test	Todas las pruebas pasan; si falla, el job termina.
Build	mvn -B package -DskipTests	target/agrosmart-monitor.jar existe.
Deploy simulado	Copiar target/*.jar a staging/	El JAR aparece en staging.
Publicacion	actions/upload-artifact@v4	Artefacto agrosmart-monitor-jar disponible.

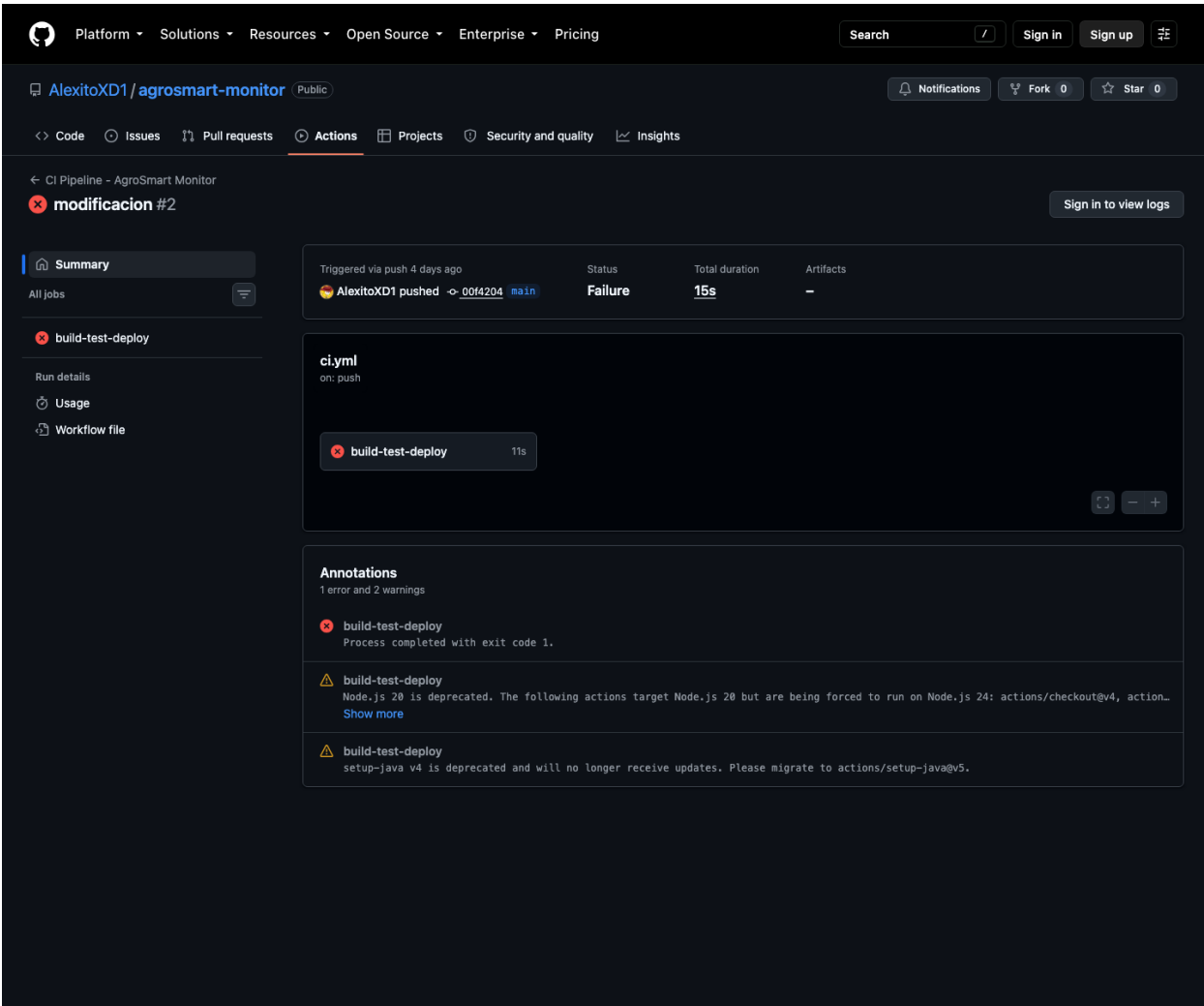
4.1 Practicas DevOps demostradas

- Control de versiones mediante commits y repositorio GitHub compartido.
- Integracion continua con ejecucion automatica en cada cambio relevante.
- Pruebas como puerta de calidad antes de empaquetar y desplegar.
- Retroalimentacion rapida por estado, anotaciones y logs del job.
- Entrega automatizada del JAR como artefacto vinculado a la ejecucion.

5. Evidencia de la ejecucion fallida

GitHub Actions #2 - Failure

URL: <https://github.com/AlexitoXD1/agrosmart-monitor/actions/runs/33288383964>
Commit: 00f4204 (modificacion)
Fecha: 29 de agosto de 2026, 21:36 GMT-5
Duracion: 15 s
Job: build-test-deploy, 11 s
Anotaciones: 1 error y 2 advertencias
Artefactos: ninguno



Captura 1. Resumen publico de GitHub Actions: estado Failure, commit 00f4204, job fallido, exit code 1 y ausencia de artefactos.

5.1 Cambio que provoco el fallo

```
assertEquals(SensorStatus.NORMAL,  
    TemperatureClassifier.classify(10.1));
```

La prueba esperaba NORMAL para 10.1 C, aunque la regla del sistema devuelve ALERT por ser una temperatura superior a 10 C. GitHub Actions registro **Process completed with exit code 1**. El problema fue una expectativa inconsistente en la prueba, no una falla del runner.

5.2 Impacto

- Maven marco la etapa de pruebas como fallida.
- Compilar y empaquetar fue omitido.
- El despliegue simulado no se ejecuto.
- No se publico ningun artefacto.
- Los pasos Post y Complete job solo limpiaron el runner; no convierten la ejecucion en exitosa.

Interpretacion

La puerta de calidad funciono: una discrepancia en la regla de negocio impidio producir y entregar una salida no validada.

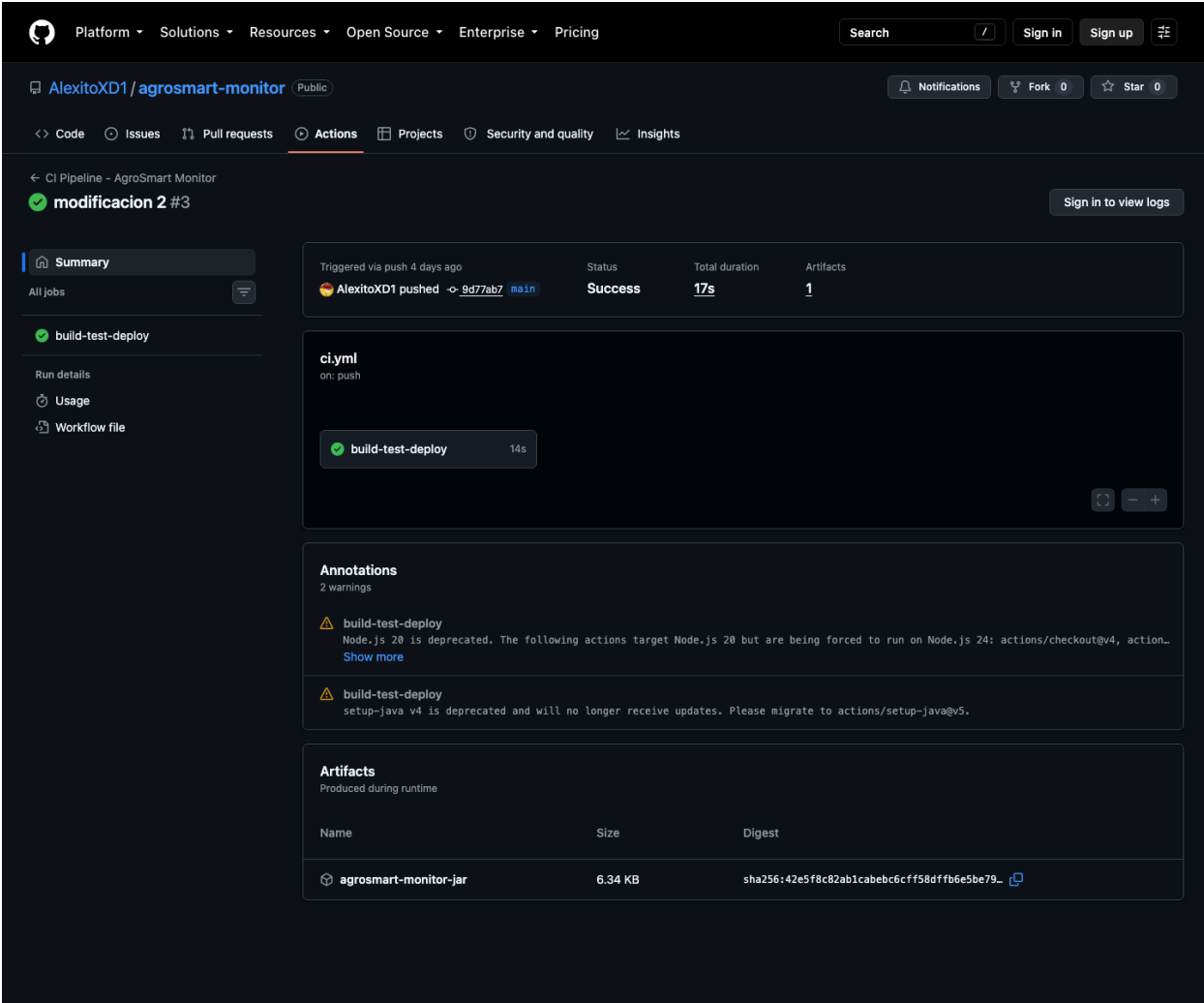
6. Correccion y evidencia exitosa

La correccion restablecio ALERT como valor esperado para 10.1 C:

```
assertEquals(SensorStatus.ALERT,  
    TemperatureClassifier.classify(10.1));
```

GitHub Actions #3 - Success

URL: <https://github.com/AlexitoXD1/agrosmart-monitor/actions/runs/33288476013>
Commit: 9d77ab7 (modificacion 2)
Fecha: 29 de agosto de 2026, 21:38 GMT-5
Duracion total: 17 s
Job: build-test-deploy, 14 s
Estado: Success
Artefacto: agrosmart-monitor-jar, 6.34 KB
Digest: sha256:42e5f8c82ab1cabebc6cff58dff6e5be795ea1d1a1a82af2b5fd86e5da60ecf



Captura 2. Resumen publico de GitHub Actions: estado Success, commit 9d77ab7, artefacto publicado y tamaño de 6.34 KB.

6.1 Secuencia de recuperacion

- 1 Corregir la expectativa para alinearla con la regla ALERT > 10 C.
- 2 Registrar el cambio en Git y enviarlo a main.
- 3 Esperar el nuevo workflow disparado por el push.
- 4 Confirmar pruebas, build, deploy simulado y publicacion del artefacto.

7. Comparacion de trazabilidad

Aspecto	Ejecucion #2	Ejecucion #3
Commit	00f4204	9d77ab7
Cambio	Esperaba NORMAL para 10.1 C	Restituyo ALERT para 10.1 C
Estado	Failure	Success
Duracion	15 s	17 s
Pruebas	Paso fallido; exit code 1	Validacion completada
Build y deploy	Omitidos	Ejecutados
Artefacto	No publicado	JAR de 6.34 KB
Efecto DevOps	Bloqueo una entrega inconsistente	Permitio entregar una salida validada

8. Checklist de entrega

Requisito	Estado	Ubicacion / evidencia
URL del repositorio	Cumplido	github.com/AlexitoXD1/agrosmart-monitor
Workflow en .github/workflows	Cumplido	.github/workflows/ci.yml
Historial de commits	Cumplido	810db2c ... 9d77ab7; incluye fallo y correccion
Ejecucion fallida explicada	Cumplido	Actions run 33288383964 / commit 00f4204
Ejecucion exitosa posterior	Cumplido	Actions run 33288476013 / commit 9d77ab7
README de ejecucion y pipeline	Cumplido	README.md

9. Conclusiones

- El pipeline conecta repositorio, pruebas, construccion y entrega automatizada.
- Una prueba fallida detuvo correctamente las etapas posteriores.
- La pareja de ejecuciones #2 y #3 demuestra el ciclo detectar, diagnosticar, corregir, validar y entregar.
- La trazabilidad queda vinculada a commits, URLs de ejecucion y al digest del artefacto.

Fuentes de evidencia

Repositorio: <https://github.com/AlexitoXD1/agrosmart-monitor>

Ejecuciones: <https://github.com/AlexitoXD1/agrosmart-monitor/actions>

Documento de referencia: TA1_Modelo_AgroSmart_Monitor_DevOps_Pipeline_CRMBQB.pdf.